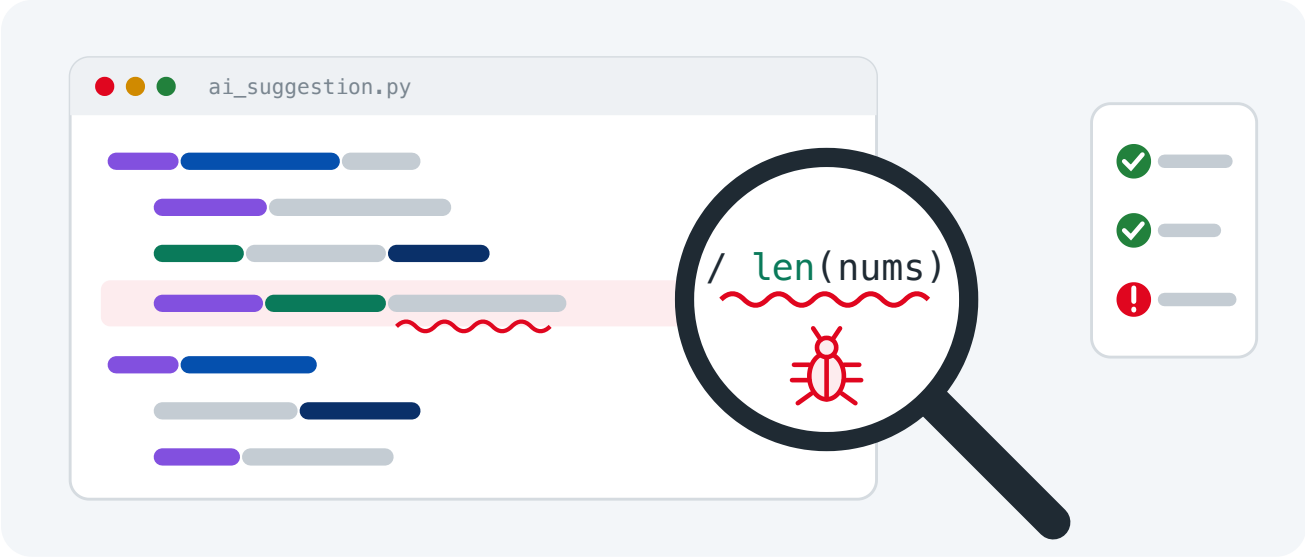
VIBE 101 · WEEK 4

Lecture 3:

Reading Code You Didn't Write

Textbook: Chapter 3 · Sections 3.1–3.8



IN THIS LECTURE YOU WILL

- Explain why reading AI-written code is its own step
- Apply the six-step reading checklist
- Trace real values to expose boundary bugs
- Recognize six common AI failure modes

> *"When an assistant writes it, understanding is a separate step that you must do deliberately."*
>
> TEXTBOOK §3.1

KEY TERMS

Spec
The written requirements the code has to meet.

Boundary
An edge input, such as an empty list, a single element, or `None`.

Trace
Following concrete values through code, one step at a time.

Code smell
A surface sign of a deeper problem, like duplicated logic or a swallowed exception.

SECTION 3.2

The Reading Checklist

When you write code yourself, you understand it by default. When an assistant writes it, you have to earn that understanding. Work through these six checks before you accept a suggestion.

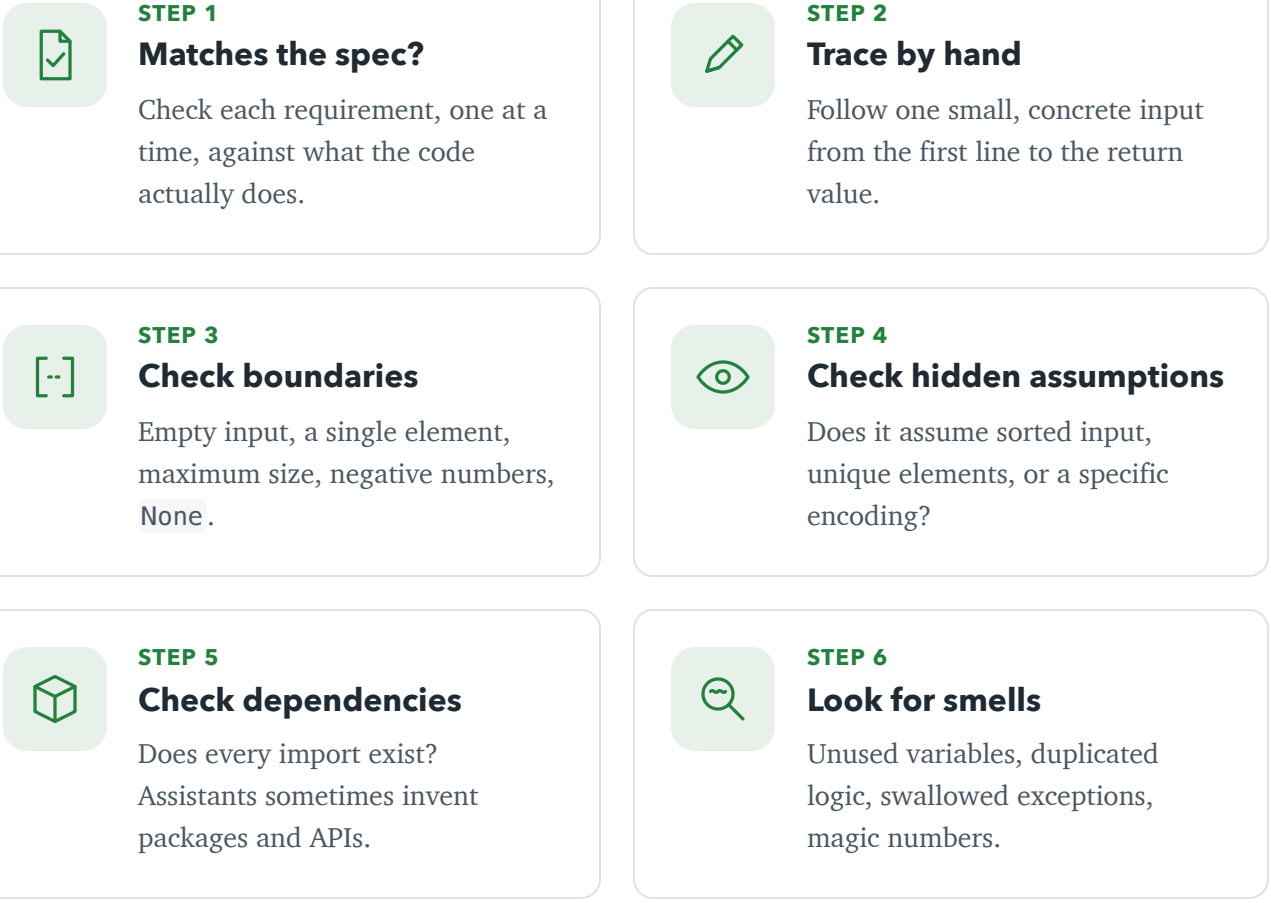


Figure 3.1 The reading checklist. Steps 1–3 catch most bugs; steps 4–6 catch the ones that only show up later, in production.

? **The ownership test.** If you can't explain why a line is there, you don't own it yet. Ask the assistant to explain it, or simplify it, before you accept the change.

WORKED EXAMPLE 3.1

Spot the Bug

An assistant suggests this function for averaging a list of numbers. It looks fine. Read it before you run it.

SUGGESTED CODE

```
def average(nums):
    return sum(nums) / len(nums)
```

TRY IT

What happens on `[]`? Trace it by hand before you read on.

STEP	average([2, 4, 6])	average([])
sum(nums)	12	0
len(nums)	3	0
sum / len	4.0	⚠ 0 / 0 → error

Figure 3.2 Trace data, not just lines. The happy-path input works. The empty list reaches a boundary the code never checks.

ANSWER

`ZeroDivisionError`. Is that in the spec? If not, it's a bug.

STEP 3 **Check boundaries.** Whether an empty list should raise an error, return a value, or never happen is a spec decision. Find it in the spec, or ask, before you accept the code.

WORKED EXAMPLE 3.2

Spot the Other Bug

This one passes a quick test. Call it once and it works perfectly. The bug only shows up the second time.

SUGGESTED CODE

```
def add_item(item, items=[]):
    items.append(item)
    return items
```

TRY IT

Call it twice. What do you get?

PYTHON SHELL

```
>>> add_item("apple")
['apple']
>>> add_item("pear")
['apple', 'pear']
```

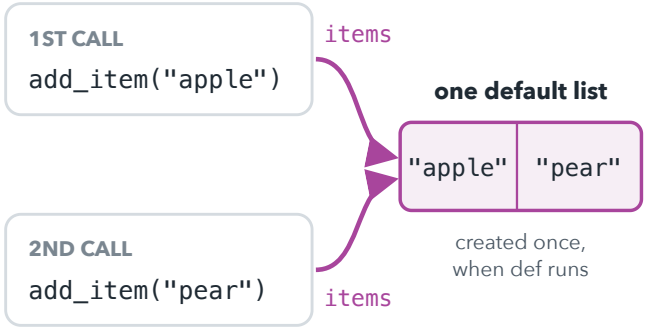


Figure 3.3 Both calls point at the same list object.

ANSWER

The default list is shared across calls. The second call returns both items.

Why it happens. Python evaluates a default value once, when the `def` statement runs, not each time the function is called. Every call that leaves out `items` reuses that same list, so changes from one call leak into the next.

STEP 2 **Trace by hand, and trace more than one call.** A single happy-path run would never show this bug.

SECTION 3.3

Common AI Failure Modes

These patterns show up again and again in assistant-written code. Each one maps to a step on the reading checklist.

Hallucinated APIs

Calls to functions or packages that do not exist.

CATCH IT **Step 5 · check the docs**

Plausible but wrong logic

Off-by-one errors and wrong comparison operators in code that reads well.

CATCH IT **Step 2 · trace by hand**

Silent `except: pass`

A bare `except: pass` hides every failure, so bugs vanish instead of surfacing.

CATCH IT **Step 6 · look for smells**

Mutable default arguments

`def f(x, items=[])` shares one list across every call.

CATCH IT **Step 2 · trace two calls**

Outdated practices

Deprecated functions and APIs from older library versions.

CATCH IT **Step 5 · check dependencies**

Overengineering

A factory for a 10-line script. Extra layers make code harder to read and review.

CATCH IT **Step 1 · check the spec**

KEY TAKEAWAYS

- Reading AI-written code is a separate, deliberate step. You own what you accept.
- Trace real values, including empty inputs and repeated calls.
- If you can't explain a line, ask or simplify before you accept it.

Next · **Lecture 4: Tests First, Vibes Second** · Read Chapter 4